

# Next-Gen Credit Intelligence

Building a Real-Time, Multi-Modal Underwriting Engine

---

*A Dual-Stream Architecture for Explainable, Verifiable Credit Decisioning*

## Synchrony Hackathon — Problem Statement 1

**Objective:** Design and develop a robust, data-driven analytics solution that expands credit access to “New-to-Credit” (NTC) and “thin-file” customers by leveraging alternative data and real-time behavioral signals, while simultaneously mitigating fraud and ensuring regulatory transparency.

**The Challenge:** Traditional credit models rely on static historical data, creating a barrier for millions of potentially creditworthy individuals who lack formal banking footprints. The goal is to shift from reactive scoring to **proactive, contextual credit decisioning**.

## Contents

|           |                                                                              |           |
|-----------|------------------------------------------------------------------------------|-----------|
| <b>1</b>  | <b>Problem Statement</b>                                                     | <b>2</b>  |
| 1.1       | Title . . . . .                                                              | 2         |
| 1.2       | Objective . . . . .                                                          | 2         |
| 1.3       | The Challenge . . . . .                                                      | 2         |
| 1.4       | Our Interpretation . . . . .                                                 | 2         |
| <b>2</b>  | <b>Solution Overview: The Dual-Stream Architecture</b>                       | <b>2</b>  |
| 2.1       | Why This Design Wins on the Rubric . . . . .                                 | 3         |
| <b>3</b>  | <b>Dataset Selection &amp; Preprocessing</b>                                 | <b>3</b>  |
| 3.1       | Why the Home Credit Default Risk Dataset . . . . .                           | 3         |
| 3.2       | Multi-Table Preprocessing Pipeline . . . . .                                 | 4         |
| <b>4</b>  | <b>Stream A: Quantitative ML Scoring Engine</b>                              | <b>5</b>  |
| 4.1       | Online Feature Engineering ( <code>features.py</code> ) . . . . .            | 5         |
| 4.2       | Leak-Safe Categorical Encoding ( <code>KFoldTargetEncoder</code> ) . . . . . | 6         |
| 4.3       | Model Architecture ( <code>DualModelScoringEngine</code> ) . . . . .         | 6         |
| 4.4       | SHAP Additivity Self-Check . . . . .                                         | 7         |
| 4.5       | Explainability: TreeSHAP Feature Families . . . . .                          | 7         |
| <b>5</b>  | <b>Stream B: Qualitative Retrieval-Augmented Generation</b>                  | <b>8</b>  |
| <b>6</b>  | <b>LLM Synthesis &amp; the Deterministic Verifier</b>                        | <b>8</b>  |
| 6.1       | Structured Output Contract . . . . .                                         | 8         |
| 6.2       | Prompt Design ( <code>build_prompt</code> ) . . . . .                        | 8         |
| 6.3       | Swappable LLM Clients ( <code>bedrock.py</code> ) . . . . .                  | 9         |
| 6.4       | Why Mock LLM, Not Live AWS Bedrock . . . . .                                 | 9         |
| 6.5       | The Deterministic Verifier . . . . .                                         | 9         |
| <b>7</b>  | <b>Responsible AI: Counterfactual Recourse</b>                               | <b>10</b> |
| <b>8</b>  | <b>Backend Service Implementation (<code>main.py</code>)</b>                 | <b>10</b> |
| <b>9</b>  | <b>Production Architecture</b>                                               | <b>11</b> |
| 9.1       | Tech Stack . . . . .                                                         | 11        |
| 9.2       | API Design . . . . .                                                         | 11        |
| 9.3       | Frontend . . . . .                                                           | 12        |
| <b>10</b> | <b>Frontend Implementation: v0.dev-Generated Risk Console</b>                | <b>12</b> |
| 10.1      | The Generation Prompt . . . . .                                              | 12        |
| 10.2      | Result . . . . .                                                             | 13        |
| <b>11</b> | <b>Rubric Alignment</b>                                                      | <b>13</b> |
| 11.1      | Design & Architecture . . . . .                                              | 13        |
| 11.2      | Engineering Practices . . . . .                                              | 14        |
| <b>12</b> | <b>Conclusion &amp; Future Work</b>                                          | <b>14</b> |

# 1 Problem Statement

## 1.1 Title

Next-Gen Credit Intelligence: Building a Real-Time, Multi-Modal Underwriting Engine.

## 1.2 Objective

To design and develop a robust, data-driven analytics solution that expands credit access to “New-to-Credit” (NTC) and “thin-file” customers by leveraging alternative data and real-time behavioral signals, while simultaneously mitigating fraud and ensuring regulatory transparency.

## 1.3 The Challenge

Traditional credit models rely almost exclusively on static historical bureau data. This creates a structural barrier for millions of potentially creditworthy individuals who lack a formal banking footprint. In a fast-evolving BFSI landscape, the goal is to move away from reactive, black-box scoring and toward **proactive, contextual credit decisioning** – one that is data-rich, explainable, and auditable at the point of decision.

## 1.4 Our Interpretation

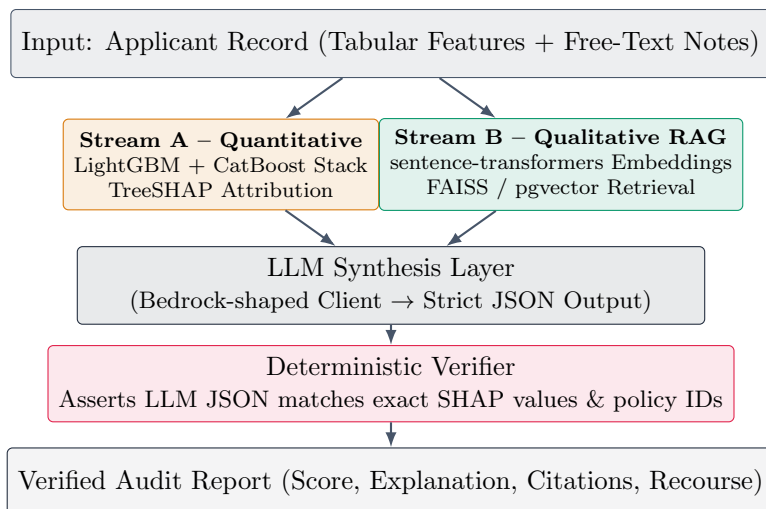
Two problems sit inside this single statement, and they demand different tools:

- **A quantitative problem** – scoring risk accurately from thin, alternative, and behavioral data.
- **A qualitative/regulatory problem** – explaining *why* a decision was made, in language a compliance officer and an applicant can both trust, without letting a language model quietly invent the reasoning.

We solve both with a single system: a **Dual-Stream Architecture** that keeps deterministic machine learning and generative explanation strictly separated, and reunites them only behind a verifier that refuses to release anything it cannot prove.

# 2 Solution Overview: The Dual-Stream Architecture

The core design principle is **separation of concerns between what decides and what explains**. A gradient-boosted ensemble computes the risk score – this is deterministic, reproducible, and fully attributable via TreeSHAP. A large language model, called through a Bedrock-shaped client interface, only *narrates* that score using retrieved policy context – it never sees raw applicant data and never determines the outcome. A final **Deterministic Verifier** cross-checks the LLM’s narrative against the exact SHAP values and retrieved policy IDs before anything reaches the user, and blocks the response if it cannot verify every claim.



## 2.1 Why This Design Wins on the Rubric

- **Explainability is mathematical, not narrative.** The LLM never invents a reason – it is handed the exact SHAP feature-family attributions and must cite them verbatim; the verifier enforces this.
- **Fails closed, not open.** If the LLM hallucinates a policy ID or a number, the system returns a blocked decision (HTTP 422), never a plausible-looking but false one.
- **Alternative data is first-class.** Thin-file signals (utility payments, telecom recharge frequency, null-counts) feed Stream A directly, addressing the NTC objective head-on.
- **Responsible AI is structural.** Protected attributes (gender, marital status, housing type) are excluded from the intake schema entirely, not merely down-weighted.

## 3 Dataset Selection & Preprocessing

### 3.1 Why the Home Credit Default Risk Dataset

The problem statement asks for a model that scores “New-to-Credit” and thin-file applicants using alternative and behavioral data rather than a conventional bureau file. We selected Kaggle’s **Home Credit Default Risk** competition dataset as the closest large, real-world analogue available:

- **Applicants without a deep credit history.** Home Credit’s own stated business problem – lending to clients with little or no formal credit history – is a direct match for the NTC/thin-file objective, rather than a generic bureau-score dataset that assumes an established file.
- **Multi-table behavioral signal, not just a flat application form.** Beyond `application_train.csv`, the dataset ships five related tables (`bureau`, `bureau_balance`, `previous_application`, `POS_CASH_balance`, `credit_card_balance`, `installments_payments`) that let us engineer genuine *behavioral* features – repayment timeliness, utilization history, prior application outcomes – which is exactly the “real-time behavioral signal” the problem statement calls for.

- **A proven, well-understood baseline.** Aguiar’s “LightGBM with Simple Features” kernel is one of the most reproduced strong baselines for this competition, which gave us a validated feature-engineering and hyperparameter starting point to extend rather than a blank page – important under hackathon time constraints.
- **Rich missingness structure.** A meaningful fraction of applicants are missing `EXT_SOURCE` bureau scores, employment history, or prior credit records entirely – which mirrors thin-file conditions naturally rather than requiring us to synthesize sparsity artificially.

### 3.2 Multi-Table Preprocessing Pipeline

Before any model sees a row, every satellite table is independently aggregated to the applicant grain (`SK_ID_CURR`) and joined onto the main application table. This is a full offline preprocessing pass (`preprocess.py`), separate from the online, single-applicant feature path Stream A uses at inference time (Section 4).

- **bureau + bureau\_balance:** one-hot encoded, aggregated to `SK_ID_CURR` with min/max/mean/var/sum statistics on credit amount, overdue amount, and days-credit fields. Applying the “Active vs. Closed split” trick from top Kaggle solutions: the same aggregations are computed a second and third time restricted to `CREDIT_ACTIVE == Active` and `== Closed` rows respectively, so the model can distinguish “struggling with a currently open loan” from “historically struggled but the loan is closed.”
- **previous\_application:** the Home Credit sentinel value 365243 is replaced with NaN across all `DAYS_*` columns (identical anomaly convention to `DAYS_EMPLOYED`). An `APP_CREDIT_PERC` ratio (requested vs. approved amount) is engineered, then the same **Approved vs. Refused split** trick is applied – aggregations are computed once overall and again restricted to each contract outcome, since a client with several refused applications behaves very differently than one with a clean approval history.
- **POS\_CASH.balance and credit\_card.balance:** one-hot encoded, aggregated with days-past-due (`SK_DPD`, `SK_DPD_DEF`) statistics and a raw record count per applicant as an exposure/tenure proxy.
- **installments\_payments:** `PAYMENT_PERC` and `PAYMENT_DIFF` (paid vs. scheduled) and `DPD/DBD` (days late / days early, floored at zero so an early payment is not treated as “negative lateness”) are computed per installment, then aggregated. This table is also the source of the recency-windowed 30/90/180-day features described in Section 4.
- **Memory management.** Every aggregated table passes through a `reduce_mem_usage()` pass that downcasts integer and float columns to the smallest safe dtype (`int8/16/32` and `float32`), which matters materially once bureau, previous-application, POS, credit-card, and installment aggregations are all joined onto a single wide applicant table.
- **Dimensionality reduction on external scores.** Because `EXT_SOURCE_1/2/3` are highly correlated and each carries its own missingness pattern, a median-imputed **2-component PCA** projection (`EXT_PCA_1`, `EXT_PCA_2`) is added as a supplementary, denoised summary signal alongside the raw scores and their NaN-safe interaction features – the model sees both representations and can use whichever fits a given split.

## Offline Preprocessing — Table Fan-In

```

1 df = pd.read_csv('data/application_train.csv')
2 df = reduce_mem_usage(df)
3
4 df = df.join(process_bureau_and_balance(), how='left', on='SK_ID_CURR')
5 df = df.join(process_previous_applications(), how='left', on='SK_ID_CURR')
6 df = df.join(process_pos_cash(), how='left', on='SK_ID_CURR')
7 df = df.join(process_credit_card(), how='left', on='SK_ID_CURR')
8 df = df.join(process_installments_payments(), how='left', on='SK_ID_CURR')
9
10 df = process_main_and_pca(df) # EXT_SOURCE_1/2/3 -> EXT_PCA_1, EXT_PCA_2
11 df.to_csv('data/application_train_enriched.csv', index=False)

```

The output, `application_train_enriched.csv`, is what `DualModelScoringEngine` actually trains on in production (Section 8) – the enrichment is a one-time offline job, not something repeated per request.

## 4 Stream A: Quantitative ML Scoring Engine

### 4.1 Online Feature Engineering (`features.py`)

Stream A’s inference-time feature pipeline, `AguiarFeatureEngine`, is a fit/transform pair so a single applicant can be scored on demand while remaining consistent with how the training data was engineered. It reproduces Aguiar’s core steps and adds three blocks designed specifically for this system:

- **Anomaly handling:** the sentinel value 365243 in `DAYS_EMPLOYED` (Home Credit’s “not employed” encoding) is replaced with `NaN` rather than treated as a real 1000-year tenure value.
- **Aguiar’s domain ratio features**, reproduced verbatim, plus one addition:
  - `DAYS_EMPLOYED_PERC`, `INCOME_CREDIT_PERC`, `INCOME_PER_PERSON`, `ANNUITY_INCOME_PERC`, `PAYMENT_RATE`
  - `CREDIT_GOODS_PERC` (requested credit vs. goods price) – added to capture negative-equity-style risk not in the original kernel
- **Thin-file proxy (TOTAL\_NULLS):** counted on the *raw, pre-cleaning* row – deliberately before the `DAYS_EMPLOYED` sentinel is converted to `NaN` – so it reflects genuine missingness in what the applicant actually submitted, not an artifact manufactured by our own cleaning step.
- **EXT\_SOURCE interactions, NaN-safe:** row-wise mean/max/min/variance/product of `EXT_SOURCE_1/2/3`. Two edge cases are handled explicitly rather than trusting NumPy’s defaults: variance is forced to `NaN` when fewer than two sources are present (a single observed value has a mathematical variance of zero, which would misleadingly read as “these sources agree perfectly”), and product is forced to `NaN` on an all-missing row (NumPy’s `nanprod` otherwise returns the empty-product identity, 1, which would look like three suspiciously agreeing near-zero scores).
- **Recency-windowed installment behavior:** for each of 30D/90D/180D lookback windows on `DAYS_INSTALLMENT`, the mean and max of days-past-due and underpayment fraction are computed and joined back via `SK_ID_CURR` → value `.map()` (not a merge), which preserves the calling

frame’s row order and index exactly – required because the label column is read off this same frame downstream. A first-time applicant with no installment history takes the identical code path as one with real history (an empty-but-correctly-shaped installments frame), rather than a separate “no data” branch that could silently drift out of sync.

- **High-cardinality categoricals are deliberately *not* encoded here.** `ORGANIZATION_TYPE` and `OCCUPATION_TYPE` are extracted before one-hot encoding runs and handed back as clean string “passthrough” columns, because leak-safe out-of-fold target encoding needs visibility into the label and the fold structure – both of which belong to `scoring.py`, not the feature module.

## 4.2 Leak-Safe Categorical Encoding (KFoldTargetEncoder)

Instead of one-hot encoding `ORGANIZATION_TYPE` and `OCCUPATION_TYPE` – which would blow up the feature space with dozens of sparse columns – `scoring.py` implements a dedicated out-of-fold target encoder:

- Uses **10 encoding folds**, deliberately decoupled from the scoring engine’s own 5 *model* folds. A row’s OOF-encoded value always excludes its own encoding fold, which is what actually prevents label leakage; using more, smaller encoding folds than model folds further shrinks a subtler leakage channel where a shared fold split lets a sliver of a model-fold’s validation labels leak into training through the encoding.
- **Bayesian mean-shrinkage smoothing** (smoothing = 20) pulls rare categories back toward the global target rate instead of overfitting to a handful of rows.
- A separate `fit_full()` map, trained on every row with no held-out split, is used *exclusively* for encoding new applicants at inference time – never for encoding a training row.

## 4.3 Model Architecture (DualModelScoringEngine)

A blended ensemble of **LightGBM** and **CatBoost**, both trained on the identical numeric design matrix (CatBoost receives no native `cat_features`, by design – see below), produces the probability of default. LightGBM is configured with the Bayesian-optimized hyperparameters from Aguiar’s reference kernel:

| Hyperparameter                | Value       |
|-------------------------------|-------------|
| <code>learning_rate</code>    | 0.02        |
| <code>num_leaves</code>       | 34          |
| <code>colsample_bytree</code> | 0.9497036   |
| <code>subsample</code>        | 0.8715623   |
| <code>max_depth</code>        | 8           |
| <code>reg_alpha</code>        | 0.041545473 |
| <code>reg_lambda</code>       | 0.0735294   |
| <code>min_split_gain</code>   | 0.0222415   |
| <code>min_child_weight</code> | 39.3259775  |

Table 1: LightGBM hyperparameters (Aguiar-optimized), Stream A

A guard, `lgbm_params_for_sample()`, rescales `min_child_weight` proportionally to sample size on small development datasets: Aguiar’s 39.33 was tuned for the full 307,511-row competition set, and on a small sample it demands more per-leaf hessian mass than exists, silently collapsing

LightGBM to a constant model with all-zero SHAP values. The guard is a no-op once trained on the full enriched dataset.

**Why margin-space blending, not rank averaging.** LightGBM and CatBoost are trained per fold and their raw log-odds *margins* – not probabilities, and not rank – are averaged 50/50:

$$\text{blended\_margin} = 0.5 \cdot \text{margin}_{\text{LGBM}} + 0.5 \cdot \text{margin}_{\text{CatBoost}}$$

TreeSHAP guarantees  $\text{margin} = \text{base} + \sum(\text{SHAP})$  for each model individually. Averaging both sides of that identity across two models falls out for free:  $\text{blended\_margin} = \text{blended\_base} + \sum(\text{blended\_SHAP})$ , with *zero* approximation error. Rank averaging – a common Kaggle ensembling trick – was deliberately rejected here: it operates on order statistics with no additive relationship to either model’s SHAP output, so a rank-blended score could never be made to satisfy the Deterministic Verifier’s exact-sum requirement. The blended margin is then mapped onto the **300–850** score scale and thresholded into APPROVE / REVIEW / DECLINE bands.

#### 4.4 SHAP Additivity Self-Check

Because the verifier checks SHAP numbers to a tolerance of  $10^{-3}$ , the engine cannot ship anything merely “close.” A self-check runs at fit time (tolerance  $10^{-6}$ , tighter than the verifier’s) and again on *every single inference request*: it reconstructs  $\text{base} + \sum(\text{SHAP})$  for the blended output and raises immediately if it does not match the blended margin. This catches a SHAP/library version mismatch loudly at training time rather than shipping numbers the verifier would silently reject one request at a time in production.

#### 4.5 Explainability: TreeSHAP Feature Families

Exact TreeSHAP values are computed for both models and averaged so the resulting SHAP vector sums exactly to the blended log-odds margin. Individual engineered columns – including one-hot suffixes and target-encoded columns like OCCUPATION\_TYPE.TE – are mapped by a longest-prefix rule onto seven **business-readable feature families** that double as ECOA/Regulation B adverse-action reason categories:

| Feature Family           | Representative Columns                                       |
|--------------------------|--------------------------------------------------------------|
| External Scores          | EXT_SOURCE_1/2/3 + mean/max/min/var/product                  |
| Debt-to-Income           | Income, credit, annuity, goods price, household size, ratios |
| Employment History       | DAYS_EMPLOYED, income type, occupation, organization         |
| Demographics & Stability | Age, registration, ID/phone recency, education, housing      |
| Bureau & Social Circle   | Bureau enquiries, social-circle default counts               |
| Recent Payment Behavior  | 30/90/180-day windowed DPD and underpayment features         |
| Data Completeness        | TOTAL_NULLS (thin-file proxy)                                |

Table 2: Feature-family mapping used to keep the LLM’s working set small

Any column that does not match a known family (or a known family plus a one-hot/target-encoding suffix) falls back to a catch-all *Applicant Profile* bucket rather than being silently dropped, so the family attributions always sum to the full SHAP vector.

### Stream A – Illustrative Output

```
1 {  
2   "risk_score": 349,  
3   "probability_of_default": 0.910759,  
4   "approval_status": "DECLINE",  
5   "key_factors": [  
6     {"family": "External Scores", "contribution": 2.7731,  
7       "contribution_pct": 70.3, "direction": "increases_risk",  
8       "top_features": ["EXT_SOURCE_3", "EXT_SOURCE_2"]}  
9   ]  
10 }
```

## 5 Stream B: Qualitative Retrieval-Augmented Generation

Stream B answers the “why does policy apply here” question. Applicant free-text notes (underwriter observations, self-reported context) are embedded with a lightweight Hugging Face model, `sentence-transformers/all-MiniLM-L6-v2` (384-dimensional embeddings), and matched against a small internal corpus of underwriting policies, each carrying a stable identifier such as `UW-1.1`.

- **Prototype (Kaggle / local notebook):** a local **FAISS** index over 5 mock policies, retrieving the top-2 nearest matches per applicant.
- **Production: PostgreSQL + pgvector** replaces the local FAISS index, so policy embeddings and structured applicant data live in the same durable store and can be queried with standard SQL alongside vector similarity search.

Critically, **the scoring model never reads the notes field** – it exists purely to drive policy retrieval, keeping the quantitative decision free of unstructured, harder-to-audit text.

## 6 LLM Synthesis & the Deterministic Verifier

### 6.1 Structured Output Contract

The final audit report is defined as a strict Pydantic schema covering approval status, risk score, key factors, policy citations, adverse-action reasons, and recourse – so the LLM’s output is validated for shape before it is even semantically checked.

### 6.2 Prompt Design (`build_prompt`)

The formatter feeds the LLM exactly three things: the SHAP feature-family attributions (with exact contribution, percentage and direction per family), the retrieved policy excerpts (with IDs), and the numeric risk outputs (score, PD, fixed status, fixed limit). The prompt is explicit that the model is **not deciding the application** – score, status and limit are computed upstream and must be reproduced verbatim – and lists six hard rules directly in the prompt text: copy the fixed numbers exactly; use only the given family names and numbers in `key_factors` with no rounding or invention; cite only from the retrieved policy ID list; populate `adverse_action_reasons` whenever the decision is REVIEW/DECLINE, referencing only risk-increasing families; never reference age, gender, marital status, or a proxy for them; and respond with a single bare JSON object, no markdown fences.

### 6.3 Swappable LLM Clients (bedrock.py)

The synthesis layer is written against a small `LLMClient` protocol (`invoke(prompt, **context) -> str`) so the model behind it can be swapped without touching the endpoint or the verifier. Three implementations exist in the current codebase:

- **GeminiClient** – the real generative call, using Google’s native `google.generativeai` SDK against `gemini-3.5-flash`, with `response_mime_type` forced to `application/json` and a defensive strip of any stray `““json` code fence the model adds anyway. The API key is loaded from environment configuration (`.env`) via `python-dotenv`, never hardcoded. The module is deliberately named `bedrock.py` and its factory function `build_llm_client()` – the system was designed against an AWS Bedrock-shaped interface (a single `invoke()` call taking a prompt and returning text) so that swapping in a `boto3` Bedrock client is a drop-in change; for this hackathon build the same interface is satisfied by Gemini’s native SDK.
- **TemplateClient** – a fully deterministic, rule-based narrative generator that fills the exact same JSON schema by string-templating the upstream numbers directly, with zero model calls. This is the default (`settings.USE MOCK LLM`) for fast local iteration and CI, since it always passes the verifier by construction and costs nothing to run.
- **MockBedrockClient** – a second deterministic stand-in with an added `hallucinate` flag that deliberately corrupts a SHAP contribution, appends a fabricated policy citation (UW-9.9, never retrieved), and inflates the credit limit. This is what powers the negative-control demo endpoint described below.

### 6.4 Why Mock LLM, Not Live AWS Bedrock

For Stream B’s synthesis call, the original plan was to host the LLM invocation on **AWS Bedrock** – consistent with the problem statement’s suggested stack and with the `bedrock.py` module name and `LLMClient` interface, which are deliberately shaped around a Bedrock `invoke_model` call. Within the hackathon’s time and budget, provisioning Bedrock model access, IAM roles, and per-request cost was not the best use of the remaining hours relative to hardening the verifier and the feature pipeline, so the live call was swapped for a **mock LLM** behind the exact same interface.

In brief, the mock LLM (`TemplateClient`, with `MockBedrockClient` as a second variant used for the negative-control demo) is *not* a generative model at all – it is a small deterministic Python function that fills the same JSON schema a real Bedrock/Gemini call would return, by directly string-templating the upstream numbers: it takes the top SHAP feature families, the retrieved policy IDs, and the fixed score/status/limit, and writes the `summary`, `key_factors`, `policy_citations`, and `adverse_action_reasons` fields from them with plain string formatting. There is no external API call, no network dependency, and no cost per request, and because the values are copied rather than generated, it passes the Deterministic Verifier by construction – which made it a reliable stand-in for demoing the full pipeline end-to-end while the live-model integration remained optional future work. Because the interface is identical, dropping in a real `boto3` Bedrock client later is a configuration change, not a rewrite.

### 6.5 The Deterministic Verifier

This is the system’s core trust mechanism. Before any report is released, a pure Python function re-checks the LLM’s JSON against ground truth:

- Every cited `policy_id` must exist in the set actually retrieved by Stream B – no hallucinated citations.
- Every numeric claim (score, SHAP contribution, percentage) must match the upstream computation exactly, to the reported decimal precision.
- The narrative’s stated reasons must align with the top-ranked SHAP feature families – the model cannot cite a minor factor while ignoring the dominant one.

**Fail-closed behavior:** if any check fails, the API does not serve a best-effort or partially-correct report. It returns HTTP 422 with the specific list of failures, and the frontend renders this as a deliberate “*DECISION BLOCKED BY VERIFIER*” state – never a silently degraded answer.

## 7 Responsible AI: Counterfactual Recourse

Beyond explaining a decision, the system tells applicants what would change it. The recourse function takes the top adverse TreeSHAP feature (e.g., a missing external score, or elevated telecom missed payments), perturbs it to a realistic healthy baseline, and re-scores the applicant with the same model – producing a concrete, quantified statement such as:

*“Actionable Advice: if you reduce missed telecom payments to 0 over the next 6 months, your score will increase by 45 points, crossing the APPROVE threshold.”*

This directly targets the “Explainability and Transparency” and “Responsible AI” rubric categories: the system does not just gate-keep, it gives thin-file and NTC applicants a legible path to approval.

## 8 Backend Service Implementation (`main.py`)

The FastAPI application wires Stream A, Stream B, the LLM layer, and the verifier into the single request/response cycle described in Section 2.

**Startup.** On boot, the app initializes the database and policy repository, then prepares Stream A: if a pre-trained artifact exists at `data/model_artifact.joblib` it is loaded directly (sub-second startup); otherwise the engine trains from scratch on the enriched dataset from Section 3 and immediately persists the result with `joblib.dump()`, so the ~1-hour training pass never repeats. The LLM client and `DeterministicVerifier` are constructed once and held in app state. Every request is also stamped with an `x-request-id` (middleware), threaded through to the audit log so a specific call can be traced end-to-end.

**Endpoints.** `GET /health` reports model/policy-store readiness and LLM mode; `GET /api/v1/policies` and `GET /api/v1/model` expose the retrieval index and a model card (OOF AUC, feature count); `POST /api/v1/underwrite` is the core endpoint; `POST /api/v1/underwrite/verify-demo` is the negative-control demo described in Section 6.4.

**The /underwrite flow** runs six individually-timed stages – Stream A scoring, Stream B retrieval, recourse generation, LLM synthesis, verification, and audit logging – and each one fails independently and explicitly rather than falling through to a generic 500: no retrieved policy is a 422 (nothing to ground the decision in), an unreachable LLM is a 502, a failed verification is a 422

carrying the failure list, and a failed audit-log write is a 500 even for an otherwise-verified decision – it is simply not released. The final **AuditReport** is always built from the *deterministically expected* status/limit (recomputed from the score, exactly as the verifier does), never from the LLM's own copy of those numbers.

#### Fail-Closed Branches in /underwrite

```

1  if not policies:
2      raise HTTPException(422, "No underwriting policy could be retrieved...")
3  if not result.passed:
4      raise HTTPException(422, detail={"error": "verification_failed",
5          "failures": result.failures, "risk_score": quant.risk_score})
6  if audit_write_error is not None:
7      raise HTTPException(500, "Verified decision could not be committed "
8          "to the compliance audit trail. Not released.")

```

Compute first, verify and commit second, release last – nothing partially correct ever reaches the caller.

## 9 Production Architecture

### 9.1 Tech Stack

| Layer             | Technology                                                                    |
|-------------------|-------------------------------------------------------------------------------|
| Frontend          | React (TypeScript), Tailwind CSS, shadcn/ui                                   |
| Backend / API     | FastAPI (async, OpenAPI/Swagger-documented)                                   |
| Database          | PostgreSQL + <b>pgvector</b> extension, plus a JSONB audit-log table          |
| LLM               | Google Gemini (native SDK) behind a Bedrock-shaped <b>LLMClient</b> interface |
| Embeddings        | <b>sentence-transformers/all-MiniLM-L6-v2</b>                                 |
| ML Models         | LightGBM + CatBoost, blended in margin space; exact TreeSHAP                  |
| Model persistence | <b>joblib</b> artifact, trained once from the enriched dataset                |

Table 3: Production technology stack

### 9.2 API Design

The dual-stream pipeline converges into a single, OpenAPI-documented endpoint:

#### Core Endpoint

##### POST /api/v1/underwrite

Request: flat, snake\_case applicant payload (income, credit request, external scores, bureau activity, free-text notes). Excludes gender, marital status, and housing type by design.

Response: verified audit report (score, PD, status, key factors, policy citations, retrieved policy context, recourse, verification metadata, per-stage latency).

A companion negative-control endpoint, POST /api/v1/underwrite/verify-demo, deliberately exercises the verifier's failure path for demo purposes.

### 9.3 Frontend

The React console is covered in full in Section 10, including the generation prompt and a screenshot of the running dashboard.

## 10 Frontend Implementation: v0.dev-Generated Risk Console

Given the hackathon’s time budget, the React frontend was not hand-coded from scratch. Instead, we wrote a single, highly detailed specification prompt – pinning down exact colors, component structure, and, critically, the *exact* API response field names – and generated the dashboard with **v0.dev**. Pasting the literal JSON response shape into the prompt (rather than describing it in prose) was deliberate: generators reliably invent their own field names otherwise, and nothing binds to the real API without a rewrite.

### 10.1 The Generation Prompt

The prompt specified, in full:

- **Visual direction:** a dark-slate enterprise console (**bg-slate-950** background, **bg-slate-900** cards, **border-slate-800**, no gradients or glassmorphism), a single **amber-500** accent for primary actions and the score dial, and decisions reserved to three semantic colors (emerald = APPROVE, amber = REVIEW, rose = DECLINE). All numbers, policy IDs and SHAP values render in **font-mono tabular-nums** so they never reflow.
- **Layout:** a fixed top bar plus a three-column grid (360px / flexible / 400px) that collapses to a single stacked column below 1g, each column scrolling independently.
- **Left column – Application Intake:** a react-hook-form + zod form covering income, requested credit, annuity, household size, an age slider that submits **DAYS\_BIRTH** as a negative day count, a not-currently-employed switch that submits Home Credit’s **365243** sentinel, three external-score sliders each with a “not available” checkbox that submits **null**, bureau/social-circle counts, education/income-type/occupation selects, and a required underwriter-notes textarea – with an explicit note that gender, marital status and housing type are excluded from the intake UI by design.
- **Center column – Risk Assessment:** a hand-built SVG semicircular score dial (300–850, no chart library) colored by decision band with an animated count-up; a Recharts horizontal bar chart of **key\_factors** with a zero reference line, per-bar coloring by **direction**, and a custom tooltip showing SHAP contributions to four decimals; and an amber-bordered *Actionable Recourse* card rendering the recourse object’s advice, current→target value, and score-delta pill.
- **Right column – Compliance Audit Trail:** a prominent green *VERIFIED* banner (or a rose *DECISION BLOCKED BY VERIFIER* alert on a 422) as the single most important element on the page, followed by an always-open accordion covering the decision summary, adverse-action reasons, policy citations with similarity bars, full retrieved-policy text, and a per-stage latency table.
- **API integration:** bound directly to **POST /api/v1/underwrite** using the exact response field names from Section 9.2, with a staged (not spinner-only) loading sequence across the four pipeline stages, distinct handling for a 422 verifier block versus a 502 unreachable-LLM error, and a “Demo: Verifier” button wired to the negative-control endpoint.

- **Engineering requirements:** typed API interfaces with no `any`, a single `useUnderwrite()` hook owning loading/data/error state, `Intl.NumberFormat` currency formatting, SHAP values always rendered to exactly four decimals, and `aria-live="polite"` on the verification banner and score dial.

## 10.2 Result

The generated console matches the specification closely on the first pass – three independently-scrolling columns, the semicircular score dial with tick marks at the REVIEW/APPROVE thresholds, a diverging TreeSHAP bar chart, an amber recourse card with a point-delta pill, and a right-hand compliance panel led by a green verification banner.

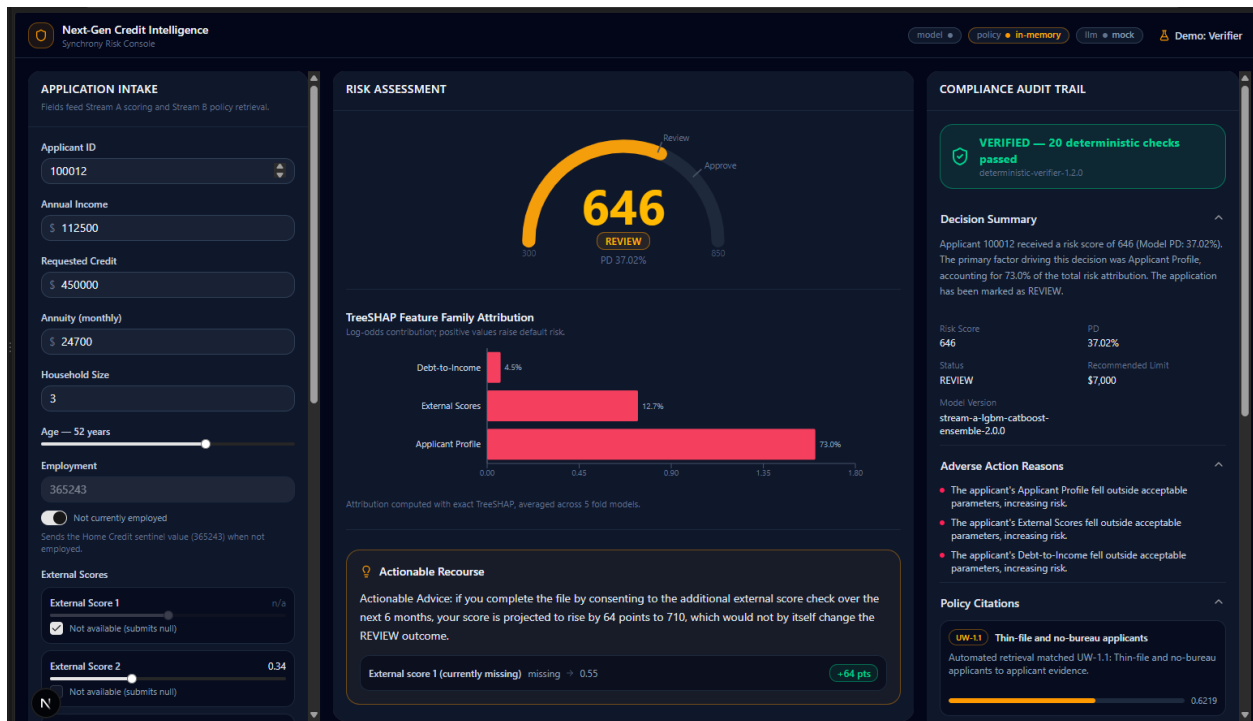


Figure 1: The v0.dev-generated console, live against a REVIEW-band applicant: score dial, TreeSHAP feature-family attribution, actionable recourse, and the verified compliance audit trail.

## 11 Rubric Alignment

### 11.1 Design & Architecture

- **Frontend (React-based UI):** A complete risk console capturing alternative-data inputs and visualizing score, TreeSHAP attribution, and the verifier's audit trail.
- **Backend (API service):** FastAPI as the modern equivalent of a Spring Boot service – async request handling, automatic OpenAPI documentation, strict Pydantic validation, per-stage timing, and a request-ID middleware for traceability.
- **Database:** PostgreSQL with the `pgvector` extension stores both structured applicant data and policy embeddings in one durable system, alongside a JSONB compliance audit log.

- **AI Layer:** all-MiniLM-L6-v2 embeddings power retrieval; the LLM sits behind a Bedrock-shaped `LLMClient` interface – built to swap in a real `boto3` Bedrock client with no changes to the endpoint or verifier – currently satisfied by Google Gemini’s native SDK, with a fully deterministic template-based client available for cost-free local iteration.
- **Security:** Pydantic-enforced input validation (surfacing as clean 422s on bad input) and secrets isolated to environment configuration (`.env` via `python-dotenv`), never hardcoded.

## 11.2 Engineering Practices

- **Explainability & Transparency:** exact TreeSHAP attribution, averaged across a two-model ensemble, summing precisely to the blended log-odds margin – not an approximation.
- **Responsible AI:** protected attributes are excluded from the schema entirely; counterfactual recourse gives applicants an actionable path forward.
- **Clean, Modular, API-First Code:** logic is separated into `features.py`, `scoring.py`, `bedrock.py`, `verifier.py`, and `database.py`, converging into one endpoint.
- **Deterministic Guardrails:** the verifier fails closed on any hallucinated policy ID, altered SHAP number, or misaligned reasoning – returning a structured 422 rather than an unverified answer.

## 12 Conclusion & Future Work

This system demonstrates that expanding credit access to thin-file and NTC customers does not require trading away transparency. By strictly separating a deterministic scoring engine from a generative explanation layer – and refusing to release anything the verifier cannot prove – the architecture is both more inclusive (through alternative-data feature engineering) and more auditable (through exact SHAP attribution and policy-grounded citations) than either a pure black-box model or an unguarded LLM report.

**Future directions** include: fairness-metric monitoring across demographic proxies not used in scoring, online/streaming feature updates for true real-time behavioral signals, and expanding the policy corpus with versioned, timestamped underwriting guidelines to support full regulatory audit trails.